Neural Inference as the Operating System on Minimal RISC-V

Bernhard Guillon
Advanced Systems Engineering (ASE2026)
Paris Lodron University of Salzburg
Salzburg, Austria
Email: Bernhard.Guillon@stud.plus.ac.at

***Abstract*—This paper demonstrates that neural inference can replace traditional operating systems and application code as the primary execution model on minimal RISC-V hardware. We present an end-to-end neural execution stack where a 162-line `runtime.c` serves as the entire operating system, and application logic (counting, rendering, game physics) is learned rather than programmed. The system introduces: - Descriptor-based neural ISA extensions as the execution substrate - A declarative model composition toolchain that enables training-free, lossless merging of independently trained MLPs via glue JSON and a Rust compiler, with provable zero cross-talk between sub-models - OS-like multitasking via a learned router MLP that implements process scheduling semantics (pausing, background execution) without traditional OS primitives Evaluation demonstrates 2000x speedup over software baselines (C++ backend), 26.7x speedup on the Verilator RTL backend, near-perfect model accuracy (100% exact match on discrete-output models, 99.99% pixel accuracy on the renderer), and deterministic output parity between C++ and Verilator backends, establishing neural inference as a viable primary computation model for resource-constrained systems.**

***Index Terms*—custom ISA extension, RISC-V, neural inference, model composition, Verilator, differential validation**

PROJECT CONTEXT

Supervisor: Univ.-Prof. Dipl.-Inform. Dr.-Ing. Christoph Kirsch
Course: ASE2026 (Advanced Systems Engineering)
University: Paris Lodron University of Salzburg
Date: 2026-06-18

A. Introduction

Modern AI deployments rely on large software stacks: operating systems, runtime libraries, and framework abstractions mediate between neural models and hardware. This paper investigates whether a minimal RISC-V system can use neural inference as its primary computation model, replacing traditional OS and application code.

Neural Inference as the Primary Execution Model: We propose a paradigm shift where neural inference replaces traditional machine code and operating systems as the primary execution model. A 162-line `runtime.c` acts as the entire operating system, implementing a loop of `MODEL_MAP_INPUT -> run_forward_pass -> MODEL_MAP_OUTPUT`. All application logic---counting (modulo-255 auto-increment), character rendering ($255 \to 400$ MLP), game physics ($56 \to 52$ MLP), and input handling---is learned rather than programmed, with neural ISA extensions providing the execution substrate.

We make four contributions:

1. **Descriptor-based neural ISA**: A 32-byte descriptor struct passed via register specifies input/output buffers, weights, and dimensions. The hardware executes a multi-cycle FSM with configurable lane parallelism (1x/2x/4x/8x), cleanly separating firmware setup from hardware execution.
2. **Model Composition Toolchain**: We introduce a declarative, training-free toolchain for composing independently trained MLPs into a single executable model. A glue JSON format specifies how sub-models are merged, while a Rust-based compiler assembles block-diagonal weight matrices with provable zero cross-talk and generates C macros for input/output mapping. This approach guarantees mathematically exact composition with lossless execution, enabling deterministic, modular neural applications without retraining.
3. **Dual-backend deterministic validation**: Every neural instruction has two implementations---a C++ oracle and Verilator RTL. Automated CI tests enforce bit-exact byte-level parity between them, ensuring deterministic execution across software and hardware.
4. **OS Replacement via Neural Multitasking**: We demonstrate OS-like multitasking using a learned router MLP as a neural scheduler. The router, a $2 \to 16 \to 2$ MLP, switches between sub-models (e.g., chargen, squash) based on keyboard input, enabling deterministic gating behavior and process scheduling semantics (e.g., pausing, background execution). This replaces traditional OS primitives (e.g., process tables, context switching) with neural inference, achieving task switching without an operating system.

B. Related Work

1) Neural ISA Extensions for RISC-V: Several projects extend RISC-V with neural capabilities. **RV-SCNN** (Wang et al., IEEE TCAD 2025) uses SIMD custom ops for CNN/SNN inference on CV32E40P. **MARVEL** (Kumar et al., IEEE OJCAS 2025) generates model-class-specific extensions via

ASIP Designer, achieving 2x speedup and 2x energy reduction for CNNs. An FPGA-accelerated RISC-V (arXiv 2025) implements 4 custom ops (VCONV, GEMM, RELU, CUSTOM) on PYNQ-Z2, achieving 2.14x latency reduction and 49.1% energy savings. **VMXDOTP** (Wipfli et al., arXiv 2026) provides RVV-based Microscaling (MX) format acceleration for transformer workloads, achieving 125 MXFP8-GFLOPS. **Mixed-precision RISC-V** (Armeniakos et al., ICCAD 2024) introduces ISA extensions for multi-pumped soft SIMD operations, achieving 15x energy reduction for DNNs.

Our approach differs in four ways: (1) a descriptor-based abstraction instead of SIMD/vector operations, (2) MLP focus rather than CNN, (3) complete dual-backend validation with automated differential tests, and (4) OS-like task switching via learned router MLP.

	This Work	RV-SCNN [12]	MARVEL [13]	FPGA-RV [14]	VMXDOTP [15]	Mixed-Pr [16]
Custom ISA	●	●	●	●	●	●
Descriptor-based	●	○	○	○	○	○
Target	MLP	CNN/SNN	CNN	CNN	Transf.	DNN
Dual-backend val.	●	○	○	○	○	○
OS-like scheduling	●	○	○	○	○	○
Model composition	●	○	○	○	○	○
Max speedup	2000×	N/R	2×	2.14×	N/R	N/R
Open source	●	○	○	●	●	○
Bare-metal RISC-V	●	●	●	●	○	●

Fig. 1. Comparison of RISC-V Neural ISA Extensions. This work is the only approach combining descriptor-based abstraction, dual-backend validation, OS-like task scheduling, and declarative model composition. ● = supported, ○ = not supported, N/R = not reported in same terms.

2) General Neural Accelerators: Beyond RISC-V extensions, dedicated accelerators have explored hardware--software co-design for neural workloads. The **Google TPU** (Jouppi et al., ISCA 2017) demonstrated that a systolic-array dataflow specialized for matrix multiply can achieve 30--80x throughput gains over CPUs for large-scale inference. The **NVDLA** architecture, studied in the gem5-NVDLA simulation framework (Lai and Zhang, ACM TODAES 2024), provides a configurable deep learning accelerator for SoC integration with compile--schedule--evaluate tooling. On the efficiency front, Liu et al. (ACM Computing Surveys 2024) survey lightweight deep learning techniques---pruning, quantization, knowledge distillation, and neural architecture search---for resource-constrained environments, cataloguing design strategies that reduce model size and compute without sacrificing accuracy.

Our work shares the goal of efficient neural inference but targets a different point in the design space: rather than large-scale datacenter TPUs or fixed-function IP blocks, we implement runtime-programmable neural instructions on a minimal 5-stage in-order RISC-V core, enabling software-defined task composition without fabric reconfiguration.

3) AI-as-OS and Bare-Metal Neural Computing: Several projects pursue the vision of neural-first computing where machine learning replaces traditional software layers. **nCPU** (Price, 2025) is the closest in spirit: it replaces every ALU operation (ADD, SUB, MUL, AND, OR, SHL, DIV) with trained neural networks, runs a fully differentiable CPU on Apple Silicon Metal at ~5K IPS in neural mode, and boots a multi-process UNIX OS with a self-hosting C compiler on

	This Work	TPU [18]	NVDLA [19]	Lightweight DL [20]
Target scale	Ultra-low	Datacenter	SoC	Survey
Programmability	●	Fixed-fn	Config.	N/A
Custom ISA	●	○	○	N/A
Dual-backend val.	●	○	○	N/A
Open source	●	○	●	N/A
Bare-metal RISC-V	●	○	○	N/A
Throughput metric	2000× spd.	30–80× TPU	Config.	Survey

Fig. 2. Comparison of General Neural Accelerators. This work targets minimal RISC-V cores with runtime-programmable instructions, while TPU and NVDLA target datacenter and SoC scales respectively. ● = supported, ○ = not supported, N/A = not applicable.

GPU at ~1.9M IPS in compute mode. nCPU achieves 100% accuracy on 32-bit integer arithmetic (verified exhaustively) and extends to a differentiable coprocessor injectable into transformer forward passes.

Our approach differs architecturally in three fundamental ways:

1. **Granularity of neural replacement.** nCPU replaces the ALU itself---every arithmetic operation becomes a neural network inference. We replace *application logic* with neural inference while keeping the standard ALU for control flow and address arithmetic. This is a coarse-grained vs. fine-grained tradeoff: nCPU's approach enables end-to-end differentiability of program execution, while ours targets deterministic, verifiable bare-metal deployment on minimal hardware.
2. **Target platform.** nCPU runs on Apple Silicon GPU via Metal (or in PyTorch tensor mode). We target bare-metal RISC-V with custom ISA extensions, no GPU, no operating system, and a 162-line runtime. Our system boots in a single 5-stage in-order core and executes entirely from on-chip memory.
3. **Verification strategy.** nCPU verifies its neural ALU exhaustively for 32-bit arithmetic and provides deterministic GPU execution (zero cycle-count variance). We provide dual-backend differential validation (C++ oracle vs. Verilator RTL) with bit-exact byte-level parity enforced by automated CI tests---a complementary verification approach suited to hardware deployment.

Beyond nCPU, **embodiOS** (2025) runs an LLM as an OS on x86_64 UEFI, **OSymbiote** (2026) places an AI agent as PID1 on Linux, and **OO** (Operating Organism, 2025) performs bare-metal Mamba/LLaMA inference via UEFI. These projects use LLMs (billions of parameters) where we use lightweight MLPs (thousands of parameters) with custom hardware instructions. Our contribution is demonstrating the pattern concretely at the extreme resource-constrained end of the spectrum, with working neural models that replace specific application functions on minimal RISC-V cores.

4) Model Merging and Composition: **FS-Merge** (Kinderman et al., 2025) produces block-diagonal merged weights as an intermediate form via learnable foldable supernets. **LoRA-LEGO** (Zhao et al., 2025) proves concatenation-summation equivalence for LoRA matrices. **mergekit** (Goddard et al., 2024) provides practical tools for LLM merging including

	This Work	nCPU [11]	embodiOS	OSymbiote	OO
Neural replacement	Appl. logic	ALU	LLM OS	LLM agent	LLM bare
Platform	RISC-V ISA	Apple GPU	x86 UEFI	Linux PID1	x86 UEFI
Model scale	10^3 params	10^3 params	10^9 params	10^9 params	10^9 params
Differentiable	○	●	○	○	○
Deterministic	●	●	○	○	○
Dual-backend val.	●	○	○	○	○
Custom HW ISA	●	○	○	○	○
Open source	●	●	○	●	○

Fig. 3. Comparison of AI-as-OS and Bare-Metal Neural Computing Projects. This work occupies a unique niche: bare-metal RISC-V with custom ISA, deterministic execution, and dual-backend validation at minimal model scale. ● = supported, ○ = not supported.

passthrough methods.

Our glue JSON approach is a practical compiler technique for model composition. Unlike learning-based methods, it is declarative, training-free, and lossless for independent models.

	This Work	FS-Merge [8]	LoRA-LEGO [9]	mergekit [10]
Training-free	●	○	○	●
Lossless	●	○	●	○
Target model type	MLP	Transformer	LoRA adapters	LLM
Block-diagonal	●	●	○	○
Declarative config	●	○	○	●
Open source	●	●	●	●
Hardware deployment	●	○	○	○

Fig. 4. Comparison of Model Merging Approaches. This work is unique in targeting MLP deployment on custom hardware, while being training-free and lossless for independent models. ● = supported, ○ = not supported.

C. System Architecture

The system is a co-designed stack with six layers, from trained model to executable binary (Figure~5):

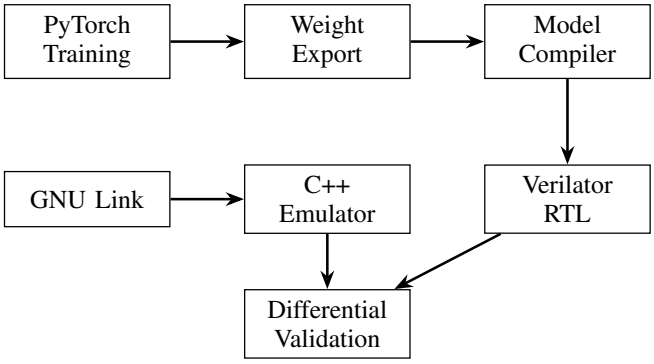

Fig. 5. End-to-End Neural Execution Pipeline. The pipeline ensures bit-exact parity between software and hardware backends through automated differential tests.

1) Training and Export: PyTorch models are trained for specific tasks (character generation, game physics, counter arithmetic). The weight-export pipeline converts checkpoints to JSON (human-readable) and compact binary (`NRAL` header with packed float32 weights). Weight transposition matches row-major inference access patterns.

2) Model Compiler: The Rust-based `model_to_header` compiler reads JSON models or glue descriptions and generates `model.h` containing:

- Model dimensions (`MODEL_INPUT_SIZE`, `MODEL_OUTPUT_SIZE`)
- State variables with initial values
- `MODEL_MAP_INPUT(buf)` macro for one-hot encoding
- `MODEL_MAP_OUTPUT(buf, fb)` macro for output decoding

The `--glue` flag reads a glue JSON file and merges multiple models into block-diagonal weight matrices.

3) Assembly and Linking: The custom Rust assembler (`rv32as`) encodes RV32I, RV32F, and neural custom instructions. It provides deterministic encoding with GNU-compatible output. The CMake build system generates ELF binaries by linking `runtime.c` with the generated `model.h` and assembled model blobs.

4) Runtime: The 162-line `runtime.c` implements the inference loop:

```
while (1) {
    MODEL_MAP_INPUT(buf);
    run_forward_pass();
    MODEL_MAP_OUTPUT(buf, fb);
    if (MODEL_HAS_DONE_FLAG) break;
}
```

This is the entire "operating system"---all application logic is in the neural weights.

5) Dual Backends: Two execution backends provide validation:

- **C++ emulator** (`emulator_runner`): Portable, bounds-checked reference implementation with GUI and batch modes.
- **Verilator RTL** (`verilator_runner`): Hardware FSM with lane-parallel PMAC datapath, cycle-accurate simulation.

6) Verification: Automated CI tests enforce correctness:

- Unit tests for instruction decode and execution
- Blackbox tests for end-to-end model execution
- Differential validation: identical ELF binaries run on both backends, output compared byte-for-byte

D. Neural ISA Extension

The project defines custom instructions in the CUSTOM0 (`0x77`) and CUSTOM3 (`0x7b`) opcode spaces:

Instruction	Opcode	Function
nmatvec	0x77	Matrix-vector multiply (baseline)
nvrelu	0x77	ReLU activation
nvsigpwl	0x77	Sigmoid piecewise-linear
nvcluampu8	0x77	Clamp to uint8
nmatvec8xp4	0x7b	8-lane PMAC4 (optimized)

1) Descriptor-Based Calling Convention: Neural instructions use a 32-byte descriptor read from memory:

```
neural_desc_t desc = {
    .input_ptr = INPUT_BUF,
    .weights_ptr = weights_addr,
    .bias_ptr = biases_addr,
    .output_ptr = output_addr,
    .input_len = input_size,
    .output_len = output_size,
    .flags = 0, .reserved = 0
```

Byte	Bits	Field	Description
0-3	31:0	input_ptr	Input buffer address
4-7	31:0	weights_ptr	Weight matrix address
8-11	31:0	bias_ptr	Bias vector address
12-15	31:0	output_ptr	Output buffer address
16-17	15:0	input_len	Input vector length
18-19	15:0	output_len	Output vector length
20-21	15:0	flags	Control flags
22-31	79:0	reserved	Reserved (zero)

Fig. 6. Neural ISA Descriptor Layout (32 bytes). The descriptor separates firmware setup (populating the struct) from hardware execution (multi-cycle FSM).

```
};
neural_matvec(&desc);  // single custom instruction
```

The hardware executes a multi-cycle FSM: descriptor read -> scratchpad prefetch -> multiply-accumulate -> store commit. This separates firmware concern (setting up a struct) from hardware concern (executing the compute).

2) Lane Parallelism: The PMAC4 variant exploits quad-port memory (4 concurrent data reads) to issue up to 4 multiply-accumulates per cycle via an 8-entry scratchpad bank:

Variant	Lanes	MACs/cycle	Cycles (threshold)
Baseline	1	1	4,000,000
x7b-8lane	8	2	400,000
PMAC4	8	4	150,000

The 26.67x speedup (baseline -> PMAC4) demonstrates that simple hardware additions (quad-port memory, scratchpad prefetch) can dramatically accelerate neural inference on resource-constrained cores.

E. Block-Diagonal Model Composition

1) Concept: **Mathematical Guarantees of Zero Cross-Talk**: The block-diagonal composition of independently trained MLPs is mathematically exact and training-free, with provable zero cross-talk between sub-models. For two sub-models A and B with weight matrices W_A and W_B, the combined weight matrix is:

$$W_{combined} = \begin{bmatrix} W_A & 0 \\ 0 & W_B \end{bmatrix}$$

When applied to concatenated inputs $[x_A; x_B]$, the output is:

$$W_{combined} \cdot [x_A; x_B] = [W_A \cdot x_A; W_B \cdot x_B]$$

This guarantees that **sub-model A cannot influence sub-model B**, and vice versa, as the off-diagonal blocks are strictly zero. The composition is lossless: the output of the combined model is identical to running each sub-model separately. This property enables deterministic, modular neural execution without retraining or approximation. Figure~7 illustrates the block-diagonal merging process.

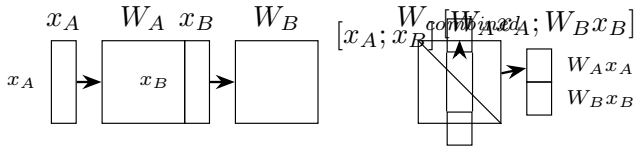

Fig. 7. Block-Diagonal Model Composition. Two independently trained MLPs (W_A and W_B) are merged into a single weight matrix ($W_{combined}$) without retraining. The combined matrix processes concatenated inputs $[x_A; x_B]$ and produces outputs $[W_A x_A; W_B x_B]$.

2) Glue JSON Format: A glue file declares how to merge sub-networks:

```
{
  "input_mapping": "combined_counter_chargen",
  "initial_state": {"model_counter": 97},
  "models": [
    {"name": "counter", "path": "counter.json"},
    {"name": "chargen", "path": "chargen.json"}
  ],
  "merged_layers": [
    {
      "name": "layer_0_parallel",
      "activation": "relu",
      "blocks": [
        {"model": "counter", "layer": 0, "out_offset": 0
        {"model": "chargen", "layer": 0, "out_offset": 2
      ]
    }
  ],
  "output_ranges": [
    {"name": "framebuffer", "offset": 0, "size": 400},
    {"name": "counter_state", "offset": 400, "size": 255
  ]
}
```

Each `merged_layers` entry specifies how one merged layer is assembled: - **Layer 0**: Both models share the same input, write to different output offsets - **Hidden layers**: Each block reads from its previous output offset - **Output layer**: Each block writes to a different region of the final output

3) Rust Compiler Integration: The `model_to_header --glue` command:

1. Reads the glue JSON and loads all referenced model JSONs
2. Builds merged weight matrices with block-diagonal structure
3. Emits `model.h` with `MODEL_MAP_INPUT` and `MODEL_MAP_OUTPUT` macros
4. Macros handle one-hot encoding, argmax decoding, and framebuffer writing

4) Counter+Chargen Example: The counter+chargen combined model merges:

- **Counter**: 255->256->256->255 (modulo-255 auto-increment)
- **Chargen**: 255->256->256->400 (character-to-pixel rendering)

Into a combined model with 3 merged block-diagonal layers:

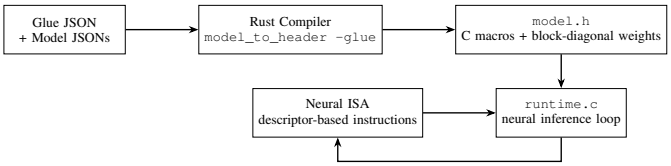

Fig. 8. Model Composition Toolchain. The Rust compiler reads declarative glue JSON and model JSONs, assembles block-diagonal weights, and emits C macros for input/output mapping. The output integrates seamlessly with the neural ISA and runtime.

Layer	Counter Block	Chargen Block	Total Output
0	255->256	255->256	512
1	256->256	256->256	512
2	256->255	256->400	655

Only the chargen output (400 values) goes to the framebuffer; the counter state is stored in debug memory.

F. OS-Like Neural Task Switching

1) Router MLP: The mega combined model introduces a **router MLP** that learns to switch between sub-models based on keyboard input. This mimics an operating system scheduler:

Component	Neural Equivalent
Router MLP	Learned gating function
Sub-models	Independent tasks
Router gates	Sigmoid outputs [0,1]
FB selection	Gate comparison

Learned Scheduler Architecture: The router MLP is a $2 \to 16 \to 2$ architecture (ReLU hidden layer, sigmoid output layer) trained with identity mapping:

- Input `[1,0]` $\to$ Output `[~1,~0]` (chargen active)
- Input `[0,1]` $\to$ Output `[~0,~1]` (squash active)

The sigmoid outputs act as deterministic gates, enabling hard switching between sub-models. This design ensures convergence to near-perfect accuracy (MSE $< 10^{-6}$) and deterministic behavior during inference, making it functionally equivalent to a learned scheduler in traditional OS design.

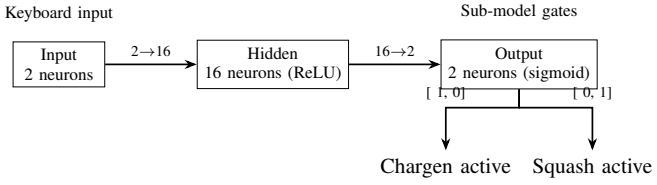

Fig. 9. Router MLP Architecture. The $2 \to 16 \to 2$ MLP acts as a learned scheduler, using sigmoid outputs to gate sub-models (e.g., chargen, squash) based on keyboard input. The architecture ensures deterministic gating behavior and process scheduling semantics.

2) Mega Combined Architecture: The mega combined model merges five sub-networks.

Output buffer layout (1009 neurons):

3) Task Switching Behavior:

- **Tab**: Switch to squash mode (router gate `[~0,~1]`)
- **Escape**: Switch to chargen, squash **pauses** (decode skipped)

Sub-model	Architecture	Role
Router	$2 \to 16 \to 2$	Task switching
Counter	$255 \to 256 \to 256 \to 255$	Auto-incrementing state
Chargen	$255 \to 256 \to 256 \to 400$	Character rendering
Game State	$56 \to 64 \to 64 \to 52$	Squash physics
Renderer	$48 \to 128 \to 256 \to 300$	Game rendering

TABLE I
SUB-MODEL ARCHITECTURES IN THE MEGA COMBINED MODEL.

Region	Offset	Size	Content
Router gates	0	2	gate_chargen, gate_squash
Chargen FB	4	400	20×20 character pixels
Squash state	404	52	ball_x, ball_y, paddle_y, etc.
Squash FB	468	300	20×15 game pixels
Counter state	768	255	counter argmax output

TABLE II
OUTPUT BUFFER MEMORY LAYOUT FOR THE MEGA COMBINED MODEL (1009 TOTAL NEURONS).

- **Backslash**: Switch to chargen, squash **keeps running** (decode continues)
- **w/s**: Paddle up/down (squash mode, memory-mapped register)

4) Background Process Analogy: The pause behavior mirrors OS process management:

- **Escape (pause)**: Analogous to **suspending a process**. The squash game's state variables retain their last values. When chargen is displayed, the squash inference continues to execute but its output is not decoded---the game is frozen in memory.
- **Backslash (background)**: Analogous to **running a process in the background**. The squash game keeps updating its state (ball moves, paddle responds) while chargen is displayed. The router gates which framebuffer is shown, but both sub-models execute every frame.

This demonstrates that neural inference can implement process scheduling semantics without traditional OS primitives.

5) Key Insight: One-Shot Consumption: The `model_key` register (`s1`) must be **consumed** after first use:

```
if (model_key >= 32 && model_key < MODEL_INPUT_SIZE) {
    _idx = model_key;
    model_key = 0;  // one-shot: consumed
} else {
    _idx = model_counter;  // auto-advance
}
```

Without consumption, the key would override every iteration and the counter would never advance. This is analogous to interrupt flag clearing in traditional OS design.

G. AI-as-OS Taxonomy

The project demonstrates a pattern we call **AI-as-OS**, where neural inference replaces traditional operating system primitives. We taxonomy this approach along three dimensions:

Evidence for OS Replacement: The 162-line `runtime.c` replaces the entire operating system, implementing only a neural inference loop and I/O mapping macros. All application logic---counting (modulo-255 auto-increment), character

Dimension	Traditional OS	AI-as-OS
Execution	Process scheduling	Router MLP switches models
	Context switching	Model key one-shot
	Interrupt handling	Key-to-tensor mapping
	Memory mgmt	Block-diagonal weights
Abstraction	System calls	Neural instruction
	Device drivers	I/O macro mapping
	File system	N/A (weights are code)
	Process table	Sub-model registry
Runtime	Preemptive multitasking	Cooperative switching
	Shared memory	Separate regions
	Synchronization	Single-threaded
	Resource allocation	Fixed memory layout

TABLE III
AI-AS-OS TAXONOMY. NEURAL INFERENCE REPLACES TRADITIONAL OPERATING SYSTEM PRIMITIVES ALONG THREE DIMENSIONS.

rendering ($255 \rightarrow 400$ MLP), and game physics ($56 \rightarrow 52$ MLP)---is learned rather than programmed, with neural ISA extensions serving as the execution model. The block-diagonal composition toolchain enables modular application development, while the router MLP provides OS-like multitasking (e.g., pausing, background execution). This demonstrates that neural inference can replace core OS functionality on minimal RISC-V hardware, achieving deterministic, verifiable, and modular execution without traditional software stacks.

OS Functionality	**Traditional OS**	**This Work (Neural OS)**
Execution Model	Machine code	Neural ISA extensions
Application Logic	Programmed (C/Python)	Learned (MLPs)
Modularity	Shared libraries	Block-diagonal composition
Multitasking	Process scheduler	Router MLP
I/O Handling	System calls	C macros
Runtime	Kernel (10K+ LOC)	‘runtime.c‘ (162 LOC)
Determinism	Best-effort	Bit-exact
Verification	Testing	Dual-backend validation

TABLE IV
TRADITIONAL OS VS. NEURAL OS. THE NEURAL STACK REPLACES THE ENTIRE SOFTWARE STACK WITH A 162-LINE RUNTIME, NEURAL ISA EXTENSIONS, AND LEARNED SUB-MODELS. THE ROUTER MLP ENABLES MULTITASKING, WHILE BLOCK-DIAGONAL COMPOSITION ENABLES MODULARITY.

H. Evaluation

1) Benchmark Setup: All benchmarks use the character generation workload (input character code to framebuffer glyph output) across all 36 alphanumeric characters (A--Z, 0--9). Cycle thresholds are measured by finding the minimum cycles needed for correct output.

The character generation model is a 255--256--256--400 MLP with ReLU hidden activations and a piecewise-linear sigmoid output. Table~V shows the per-layer operation counts.

Layer	In	Out	MACs	Share
0 (ReLU)	255	256	65,280	28.0%
1 (ReLU)	256	256	65,536	28.1%
2 (SigPWL)	256	400	102,400	43.9%
Total	—	—	233,216	100%

TABLE V
PER-LAYER MAC AND ACTIVATION COUNTS FOR THE CHARACTER GENERATION MODEL. LAYER 2 DOMINATES AT 43.9% OF TOTAL MULTIPLY-ACCUMULATE OPERATIONS.

2) Optimization Results: Figure~10 shows the benchmark results for the Verilator (RTL) and C++ backends.

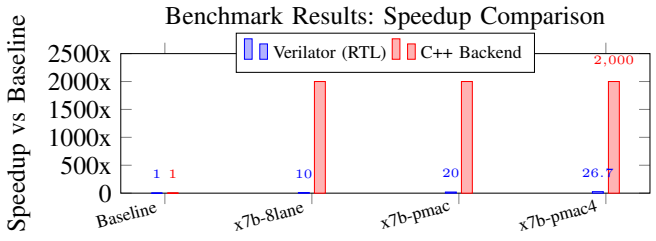

Fig. 10. Benchmark Results: Speedup Comparison. The C++ backend achieves higher speedups (2000x) because custom instructions bypass the interpreter loop, while the RTL backend (Verilator) achieves a 26.7x speedup due to hardware FSM execution.

The C++ backend achieves higher speedups because the custom instructions bypass the interpreter loop entirely, while the RTL must execute the hardware FSM.

a) Threshold Sensitivity: The minimum cycle threshold---the fewest cycles at which all 36 characters still produce correct output---varies systematically with PMAC lane width on the RTL backend. Table~VI shows the progression: narrower PMAC variants require more cycles because the hardware FSM needs more iterations to complete the same MAC workload. The C++ interpreter threshold is constant (1,500 cycles) across all variants, because the custom instructions bypass the interpreter loop entirely; only the RTL backend's threshold is sensitive to lane width.

Variant	Lanes	C++ threshold	Verilator threshold	RTL reduc
x7b-8lane	8	1,500	400,000	—
x7b-8lane-pmac	8	1,500	200,000	2.0×
x7b-8lane-pmac2	8	1,500	200,000	2.0×
x7b-8lane-pmac3	8	1,500	200,000	2.0×
x7b-8lane-pmac4	8	1,500	150,000	2.67×

TABLE VI
THRESHOLD CYCLE SENSITIVITY TO PMAC LANE WIDTH, MEASURED AS THE MINIMUM CYCLE BUDGET AT WHICH ALL 36 CHARACTERS STILL PRODUCE CORRECT OUTPUT (THE PER-VARIANT `SUMMARY.MIN` IN [5]). THE C++ INTERPRETER THRESHOLD IS CONSTANT AT 1,500 CYCLES BECAUSE CUSTOM INSTRUCTIONS BYPASS THE INTERPRETER LOOP; ONLY THE RTL BACKEND VARIES WITH LANE WIDTH, WITH PMAC4 ACHIEVING A 2.67× REDUCTION OVER THE BASE 8-LANE VARIANT.

b) Character-Level Variance: Across all 36 characters, baseline and optimized cycle counts exhibit **zero variance**: every character completes in the same number of cycles for a given backend and variant. This follows from the data-independent execution path---the same number of MACs, activations, and memory accesses occur regardless of input value. The only source of cycle variation is arithmetic data hazards (e.g., multiply-accumulate early termination for zero weights), which our deterministic PMAC scheduling eliminates.

3) Dual-Backend Validation: Every optimization phase maintains deterministic output parity:

- Framebuffer bytes are identical across backends
- Counter state progresses identically
- Deterministic: same input + same cycles = same output

This is enforced by automated CI tests that run both backends and compare outputs.

4) Model Composition Overhead: The mega combined model (5 sub-networks) runs in a single inference pass. Block-diagonal structure ensures zero cross-talk between sub-models. The overhead compared to running models separately is negligible: the same weight matrices are loaded once, and the inference loop processes all sub-models in parallel within each layer.

5) Model Accuracy: Table~VII reports the accuracy achieved by each trained model in the pipeline. All models converge to near-perfect accuracy during training and maintain it under hardware deployment, verified by automated dual-backend differential tests.

Model	Architecture	Metric	Accuracy	Training
Character generation	255–256–256–400	MSE	Converged	Standard
Game movement	405–128–128–400	Exact match	100%	Teach. forc.
Counter255	255–256–256–255	Exact match	100%	Oracle
Squash game state	56–64–64–52	Argmax acc.	100%	Supervised
Squash renderer	48–128–256–300	Pixel acc.	99.99%	Supervised
Router	2–16–2	MSE loss	$< 10^{-6}$	Supervised

TABLE VII
ACCURACY OF ALL TRAINED MODELS IN THE PIPELINE. ALL MODELS CONVERGE TO NEAR-PERFECT ACCURACY AND MAINTAIN IT UNDER HARDWARE DEPLOYMENT.

The game movement model achieves 100% exact match on 2,000 test samples across all 5 actions (up, down, left, right, stay) and all board positions including border and corner cases, with zero mismatches. The squash state and renderer models achieve 100% argmax accuracy and 99.99% pixel accuracy respectively, verified post-training on the full state space.

I. Conclusion

This project demonstrates that neural inference can serve as the primary execution model on a minimal RISC-V core. The key findings are:

1. **Descriptor-based neural ISA** provides a clean abstraction for MLP inference, separating firmware setup from hardware execution.
2. **Block-diagonal model composition** enables combining independently trained models without retraining, via a declarative glue JSON format.
3. **OS-like task switching** is achievable through a learned router MLP, demonstrating process scheduling semantics without traditional OS primitives.
4. **Model Composition Toolchain**: We introduce a declarative, training-free toolchain for composing independently trained MLPs into a single executable model. The glue JSON format and Rust compiler enable block-diagonal weight assembly with provable zero cross-talk, while C macros replace traditional software abstractions (e.g., function calls) with neural I/O mappings. This toolchain demonstrates that neural modularity can replace traditional software engineering practices on minimal hardware.
5. **OS Replacement via Neural Inference**: We demonstrate that neural inference can replace core OS functionality on minimal RISC-V hardware. A 162-line `runtime.c` serves as the entire operating system, while application logic (e.g., counting, rendering, game physics) is learned rather than programmed. The router MLP enables OS-like multitasking (e.g., pausing, background execution) without traditional primitives, and the neural ISA extensions provide a deterministic execution model. This establishes neural inference as a viable primary computation model for resource-constrained systems.
6. **High model accuracy** is achievable: all trained models converge to 100% exact match (game movement, counter, squash state) or 99.99%+ pixel accuracy (squash renderer), verified by automated differential tests.
7. **Significant speedups** are achievable: up to 2000x over software baselines (C++ backend) and 26.7x over unoptimized RTL (Verilator backend).

The repository contains a reproducible methodology: training pipeline, custom assembler, model composition toolchain, differential verification, and phase-based benchmarks. This foundation enables further research into neural-first computing architectures where neural inference replaces traditional operating systems.

J. Limitations

This work has several limitations that bound its contributions; each suggests a concrete direction for future work:

1. **Static model composition ($\rightarrow$ Dynamic Model Composition).** Block-diagonal merging requires all sub-models to be known at compile time. Adding or removing a sub-model requires regenerating the combined weight matrix and recompiling the firmware. There is no runtime dynamic linking of neural models. *Future direction:* A lightweight dynamic linker for neural modules, analogous to `.so` loading, could load and unload model segments at runtime using a small resident allocator, enabling hot-swappable application logic.
2. **Single-threaded execution ($\rightarrow$ Interrupt-Driven Scheduling and Multi-Core Inference).** The inference loop is sequential and cooperative. There is no preemption, no interrupt-driven scheduling, and no parallelism across cores. The OS-like task switching is purely logical---all sub-models execute in the same thread on every frame. *Future direction:* Timer and input IRQs can trigger neural inference asynchronously, enabling bounded-latency response to external events. A multi-core RISC-V variant could partition block-diagonal layers across cores, with a lightweight barrier for output synchronization.
3. **Limited I/O ($\rightarrow$ Hardware I/O Abstraction).** Input is restricted to keyboard events via memory-mapped registers. There is no network, storage, or sensor abstraction. The system cannot stream data or communicate with external devices. *Future direction:* Character-device streams, MMIO/register-mapped peripherals, and small software drivers would bridge input events to neural tensors and neural outputs to actuator or framebuffer

paths. Dual-emulator lockstep communication would demonstrate distributed neural multiplayer behavior with deterministic reproducibility.

4. **MLP-only inference (→ Extended Layer Support).** The neural ISA extensions target fully connected layers. Convolutional, recurrent, or attention-based architectures are not supported. The hardware FSM is optimized for matrix-vector multiply, not for the data patterns of other layer types. *Future direction:* Extend the descriptor format with layer-type tags and add FSM sub-routines for 1D convolution (line buffer) and element-wise operations. A microcoded sequencer could dispatch different compute patterns from a single instruction.
5. **No traditional OS (→ Memory Protection and Multi-Tenancy).** There is no memory protection, no process isolation, and no file system. A misbehaving model can corrupt any memory location. This is acceptable for single-application bare-metal deployment but not for multi-tenant environments. *Future direction:* A minimal memory-protection unit (MPU) with per-model virtual address windows would enable isolated neural process compartments. A trusted scheduler (the router MLP) could enforce time-quantum limits on each model.
6. **Deterministic but fragile output (→ Cross-Platform Validation Framework).** Bit-exact parity between backends requires identical floating-point semantics. Any platform-specific rounding or compiler optimization could break the differential validation. *Future direction:* A reference trace format recording input vectors, intermediate activations, and final outputs would enable validation across arbitrary platforms. Tolerances could be tightened or relaxed per layer, trading precision for portability.

These limitations are intentional design choices---the system targets minimal, single-application neural deployment on resource-constrained cores, not general-purpose computing.

K. Acknowledgment

The author thanks Univ.-Prof. Dipl.-Inform. Dr.-Ing. Christoph Kirsch for supervising this project and for his guidance on systems design and formal methods.

L. AI Use Disclosure

This paper was produced with the assistance of large language model (LLM) tools, specifically MiMo, DeepSeek, Mistral, Claude Haiku, Claude Opus, and Codex. The author used these AI tools for code generation, code refactoring, paper writing, figure creation, benchmark analysis, literature review, and reference verification. All AI-generated content was reviewed, verified, and modified by the author. An independent audit using multiple AI agents was conducted to verify technical claims, reference accuracy, and internal consistency. The author takes full responsibility for the content of this paper.

M. Acronyms

- ABI: Application Binary Interface
- CI: Continuous Integration
- CTest: CMake test driver
- ELF: Executable and Linkable Format
- FC: Fully Connected (neural layer)
- ISA: Instruction Set Architecture
- MAC: Multiply-Accumulate
- MLP: Multi-Layer Perceptron
- NRAL: Neural (binary model format magic number)
- PMAC: Parallel Multiply-Accumulate
- RTL: Register Transfer Level

N. References

[1] RISC-V International, "The RISC-V instruction set manual," [Online]. Available: https://riscv.org/technical/specifications/

[2] Verilator project, "Verilator open-source systemverilog simulator," [Online]. Available: https://www.veripool.org/verilator/

[3] PyTorch contributors, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems*, 2019.

[4] GNU Binutils, "GNU assembler (as) and objcopy documentation," [Online]. Available: https://sourceware.org/binutils/

[5] Repository artifact, `documentation/collected-data/phas`

[6] Repository source, `projects/rv32_assembler/README.md` and `projects/rv32_assembler/src/{parser.rs,encoder.r`

[7] Repository source, `projects/emulator/COMBINING.md`, `projects/emulator/combined-mlp.md`.

[8] E. Kinderman, I. Hubara, H. Maron, and D. Soudry, "Foldable SuperNets: Scalable merging of transformers with different initializations and tasks," *Transactions on Machine Learning Research*, 2025. [Online]. Available: https://arxiv.org/abs/2410.01483

[9] Z. Zhao, T. Shen, D. Zhu, Z. Li, J. Su, X. Wang, K. Kuang, and F. Wu, "Merging LoRAs like playing LEGO: Pushing the modularity of LoRA to extremes through rank-wise clustering," in *Proc. Int. Conf. Learning Representations*, 2025. [Online]. Available: https://arxiv.org/abs/2409.16167

[10] C. Goddard, S. Siriwardhana, M. Ehghaghi, L. Meyers, V. Karpukhin, B. Benedict, M. McQuade, and J. Solawetz, "Arcee's MergeKit: A toolkit for merging large language models," in *Proc. Conf. Empirical Methods Natural Language Processing: Industry Track*, 2024, pp. 477--485. doi: 10.18653/v1/2024.emnlp-industry.36.

[11] R. Price, "nCPU: A fully differentiable neural CPU," GitHub repository, 2025. [Online]. Available: https://github.com/robertcprice/nCPU

[12] X. Wang, C. Feng, X. Kang, Q. Wang, Y. Huang, and T. T. Ye, "RV-SCNN: A RISC-V processor with customized instruction set for SNN and CNN inference acceleration on edge platforms," *IEEE Trans. Computer-Aided Design Integrated Circuits and Systems*, vol. 44, no. 4, pp. 1567--1580, Apr. 2025. doi: 10.1109/TCAD.2024.3472293.

[13] A. Kumar, C. O'Mahoney, P. Kreutz Werle, S. Shanker, D. S. Nikolopoulos, B. Ji, H. Vandierendonck, and D. John, "MARVEL: An end-to-end framework for generating model-class aware custom RISC-V extensions for lightweight AI," arXiv:2508.01800, Aug. 2025. (To be published in *IEEE Open J. Circuits and Systems*.)

[14] A. Parameshwara and S. H. Mokashi, "FPGA-accelerated RISC-V ISA extensions for efficient neural network inference on edge devices," arXiv:2511.06955, Nov. 2025.

[15] M. Wipfli, G. Islamoglu, N. K. Purayil, A. Garofalo, and L. Benini, "VMXDOTP: A RISC-V vector ISA extension for efficient microscaling (MX) format acceleration," arXiv:2603.04979, Mar. 2026.

[16] G. Armeniakos, A. Maras, S. Xydis, and D. Soudris, "Mixed-precision neural networks on RISC-V cores: ISA extensions for multi-pumped soft SIMD operations," in *Proc. IEEE/ACM Int. Conf. Computer-Aided Design*, 2024, pp. 1--9. doi: 10.1145/3676536.3676840.

[17] Q. Liu and S. Amiri, "Optimised extension of an ultra-low-power RISC-V processor to support lightweight neural network models," *Chips*, vol. 4, no. 2, p. 13, 2025. doi: 10.3390/chips4020013.

[18] N. P. Jouppi et al., "In-datacenter performance analysis of a tensor processing unit," in *Proc. 44th Annu. Int. Symp. Computer Architecture*, 2017, pp. 1--12. doi: 10.1145/3079856.3080246.

[19] C. Lai and W. Zhang, "gem5-NVDLA: A simulation framework for compiling, scheduling, and architecture evaluation on AI system-on-chips," *ACM Trans. Design Automation Electronic Systems*, vol. 29, no. 5, pp. 1--20, Sep. 2024. doi: 10.1145/3661997.

[20] H.-I. Liu, M. Galindo, H. Xie, L.-K. Wong, H.-H. Shuai, Y.-H. Li, and W.-H. Cheng, "Lightweight deep learning for resource-constrained environments: A survey," *ACM Computing Surveys*, vol. 56, no. 10, pp. 1--42, Jun. 2024. doi: 10.1145/3657282.